Orden de trabajo y coordinación del equipo

Versión 1.0 para revisión del equipo. Actualizada el 30 de octubre de 2026.

Objetivo: saber qué puede empezar cada persona, qué debe recibir de otra y cómo entregar cambios sin editar el mismo archivo al mismo tiempo.

Esta guía complementa `docs/02_planificacion/PLAN_SDD.md` (alcance y dependencias), `docs/03_guias/GUIA_EQUIPO.md` (procedimientos) y `docs/04_integrantes/A/TAREAS_A.md`, `docs/04_integrantes/B/TAREAS_B.md`, `docs/04_integrantes/Angel_Zambrano/TAREAS_C.md` (fichas). La consigna académica define los requisitos oficiales. Si esta guía propone cambiar el alcance o un contrato, el equipo debe registrarlo primero en el plan o la spec correspondiente.

Angel Zambrano es el responsable antes identificado como C. Se conservan `docs/04_integrantes/Angel_Zambrano/TAREAS_C.md`, `GH_USER_C` y `persona-C` como identificadores internos; en GitHub se usa el usuario real de cada integrante.

1. Las cinco reglas de trabajo

1. **Una tarea tiene un responsable, un issue, una rama y un PR.** Antes de editar, registrá qué archivos vas a tocar. El equipo mantiene el mínimo acordado de tres commits con cambios reales por tarea.
2. **El dueño coordina los cambios en su área.** Para modificar un archivo ajeno, acordalo primero en el issue; quien edita queda identificado y el dueño revisa. Un aviso en el chat debe quedar respaldado en el issue o PR.
3. **Los contratos se aprueban antes del código.** Firmas, columnas, claves y reglas que afectan a otros se documentan y acuerdan con los tres integrantes.
4. **Una dependencia está lista cuando está mergeada y verificada en `main`.** Un archivo de referencia o un PR abierto permiten preparar trabajo, pero no acreditan una entrega terminada.
5. **Los cambios entran por PR.** Revisión de otra persona, verificaciones aprobadas y merge commit, sin squash ni force-push sobre historial publicado.

Antes de empezar: checklist de dos minutos

[ ] Leí el issue, la ficha y sus criterios de aceptación.

[ ] Confirmé las dependencias y qué entrega falta, si hay alguna.

[ ] Soy el responsable o tengo el acuerdo escrito del dueño.

[ ] Declaré los archivos que voy a editar y comprobé si alguien ya los reservó.

[ ] Tengo el árbol de trabajo limpio y una rama propia basada en `main` actualizado.

Si falta una dependencia, registrá el bloqueo. Podés preparar specs, tests con fixtures o un borrador contra el contrato acordado; no presentes esa integración como terminada.

Comprobar el punto de partida

El estado actual y la evidencia se consultan en avance general. No se mantiene otra foto del progreso aquí. Verificá código existente y criterios pendientes antes de crear archivos o declarar tareas cerradas; no recrees el repositorio para repetir el arranque.

2. Quién coordina cada área

Las rutas de ETL, BD, modelos y visualización parten de `src/demanda/`.

Responsable	Código y ejecución	Documentación y notebooks
A	`etl/extract.py`, `etl/transform.py`, `db/schema.sql`, `db/repository.py`, `models/logreg.py`.	Diccionario de datos, spec de datos y Logistic Regression, notebook de Logistic Regression.
B	`etl/validate.py`; `models/preprocessing.py`, `splits.py`, `evaluate.py`, `train.py`, `rf.py`; `scripts/run_all.py`.	Specs de Random Forest; notebooks de EDA, Random Forest y comparación.
Angel Zambrano	`etl/load.py`, `etl/pipeline.py`, `scripts/run_etl.py`, `models/xgb.py`, `viz/`, `tests/fixtures.py`; `.github/`, `CODEOWNERS`, dependencias y empaquetado.	Specs de XGBoost y visualización; notebook de XGBoost; guías de infraestructura y coordinación.

Cada responsable mantiene los tests de su módulo. A revisa los cambios de Angel Zambrano; Angel Zambrano revisa los de B; B revisa los de A.

Archivos que necesitan coordinación especial

Archivo o grupo	Regla para editarlo
`config/config.yaml`	Declarar sección y claves en el issue. Coordinar el turno con los otros PR abiertos. Si cambia una regla compartida, aprobar antes el contrato.
`docs/07_especificaciones/00-contratos.md`, `docs/07_especificaciones/03-comparacion.md`	Nombrar un redactor por cambio. Los otros dos revisan. Editar una sección concreta y registrar las aprobaciones.
`docs/02_planificacion/PLAN_SDD.md` y guías	Un autor por actualización. Explicar qué regla cambia y revisar coherencia con las fichas y la consigna.
`requirements.txt`, `pyproject.toml`	Solicitar el cambio a Angel Zambrano o acordar quién lo prepara. Verificar instalación y compatibilidad; una dependencia puede afectar a todos.
`docs/05_seguimiento/avance.md`	Cada persona actualiza sus tareas. Si otra persona integra un cambio conjunto, registra el acuerdo y la evidencia de cierre.
`docs/08_entrega/model_card.md`, `docs/08_entrega/informe_final.md`, diapositivas	Acordar secciones, turno de edición y quién consolida. Usar borradores separados solo si el equipo acuerda sus rutas en el plan.

Cómo reservar un archivo

En el issue escribí: **responsable, archivos/secciones, objetivo y próximo punto de entrega**. Si ya hay otro autor trabajando en el mismo archivo, acuerden quién termina primero o cómo dividir el cambio antes de editar. Actualicen la reserva al abrir el PR y libérenla al mergear o reasignar.

Caso actual: `tests/test_etl.py`**.** Contiene pruebas de varias áreas. Solo una persona lo edita a la vez. Para nuevas tareas, preferir archivos específicos como `test_load.py` o `test_xgb.py`; no reorganizar los tests existentes dentro de un PR ajeno a esa reorganización.

`CODEOWNERS` ayuda a identificar revisores; no bloquea por sí solo la edición ni reemplaza los acuerdos y aprobaciones del equipo.

3. Orden de trabajo: avanzar por entregas

Las semanas del plan son orientativas. No hace falta esperar al cierre de una semana para empezar algo desbloqueado; una demora tampoco detiene tareas independientes. Las etapas agrupan entregas: para tomar una tarea rigen sus dependencias concretas, no la finalización de toda la etapa anterior.

Etapa	Trabajo y responsables	Condición para avanzar
1. Fundación	Angel Zambrano completa T-00 y T-15. Los tres cierran T-01 y, después, CMP-1.	Acceso, entorno y CI verificados; decisiones D1-D7 y contratos aprobados; criterios de comparación fijados antes de ver resultados.
2. Piezas del ETL	A: T-02, T-03, ETL-1 y ETL-3. B: ETL-2 y MC-3. Angel Zambrano: ETL-4 y VIZ-2 después de T-03.	Piezas aprobadas en `main`. La prueba de independencia train/test cubre el método de etiqueta elegido.
3. ETL integrado	Angel Zambrano entrega ETL-5 después de ETL-1, ETL-2, ETL-3 y ETL-4.	CSV a SQLite reproducible, conteos y auditoría verificados, rechazo limpio de datos inválidos.
4. Base común	A: DB-1 y DB-2. B: MC-1, MC-2, MC-3 y MC-4. Angel Zambrano: VIZ-1 y su módulo XGBoost, según sus dependencias.	Datos, preprocesador, split/CV, evaluador, persistencia y runner funcionan con contratos comunes.
5. Modelos	A: LR-1 a LR-7. B: RF-1 a RF-7. Angel Zambrano: XG-1 a XG-7.	Runs reales, métricas, matrices y configuraciones comparables guardados; notebook e interpretación por modelo.
6. Comparación	A: CMP-2. Angel Zambrano: CMP-3. B: CMP-4. Los tres: CMP-5.	Consulta y gráficos comparables, análisis y decisión según CMP-1.
7. Cierre	Angel Zambrano: V-1. B: F-1. Los tres: F-2, F-3 y F-4.	Ejecución completa, revisión, BD accesible, presentación y ensayo de defensa.

Qué sí puede hacerse en paralelo

- **Después de T-00:** A prepara T-02; Angel Zambrano prepara T-15; los tres preparan los acuerdos de T-01.
- **Después de T-01:** A trabaja en T-03 y ETL-3; B en ETL-2 y MC-3; cada integrante escribe la spec de su modelo. ETL-1 espera T-02.
- **Después de T-03:** Angel Zambrano puede trabajar en ETL-4 y VIZ-2; A en DB-2; cada módulo de modelo espera también su spec aprobada.
- **Después de ETL-5:** A trabaja en DB-1; B en MC-1, MC-2 y ETL-6; Angel Zambrano en VIZ-1. Los baselines requieren módulo de modelo, preprocesador, split y evaluador común MC-3; los parámetros se comparan con validación temporal de train.

No iniciar tareas de B antes de T-01 como si carecieran de dependencias. Angel Zambrano no implementa LR ni RF: puede colaborar cuando el responsable lo acuerda, conservando la asignación y la revisión.

Para integrar un modelo: además de la spec y módulo propios, hace falta usar los componentes comunes. La persistencia final espera MC-4 y DB-2. CMP-2 requiere también CMP-1; no se cambian criterios al conocer qué modelo gana.

4. Ciclo de una tarea y revisión

Abrir la rama

Primero revisá `git status`. Si hay cambios pendientes, terminá y registrá ese trabajo antes de cambiar de rama. Ejemplo para ETL-4:

```
git switch main
git pull --ff-only origin main
git switch -c feat/ETL-4-load
```

El flujo es **spec, test, implementación y documentación**. Revisá cada diff y hacé commits pequeños con `tipo(ID): descripción`. En tareas documentales se verifican criterios y coherencia, sin inventar tests o commits vacíos.

Mantener tu rama actualizada

Desde tu rama y con el trabajo guardado en commits, incorporá los cambios de `main` antes de abrir el PR y si otra entrega modificó archivos que usás:

```
git fetch origin
git merge origin/main
```

Si hay conflictos, revisalos con el dueño del archivo. Podés cancelar una integración en conflicto con `git merge --abort`. No descartes archivos completos para resolver cambios de unas pocas líneas. Si el agente tocó algo fuera de alcance, inspeccioná el diff y retirá solo sus cambios, preservando el trabajo previo.

Abrir el PR

Ejecutá `pytest -q` y `ruff check src tests` para cambios de código o CI. Para documentación, verificá enlaces y coherencia; la CI del repositorio sigue siendo obligatoria cuando esté configurada. Subí la rama, enlazá el issue con `Closes #N` y explicá qué cambió y cómo lo verificaste.

Autor del PR	Revisor habitual
A	B
B	Angel Zambrano
Angel Zambrano	A
Cambio de contrato	Los otros dos integrantes

El revisor responde dentro de las próximas 24 horas o avisa cuándo podrá hacerlo. Si no puede, acuerdan otro integrante disponible, sin autoaprobarse. La referencia de tamaño es unas 400 líneas; si la tarea necesita más, justificarlo o acordar cómo dividirla sin romper sus criterios.

Cuándo está terminado

El checklist de aceptación es único: definición de hecho del plan. El checklist técnico del revisor está en la guía del equipo, sección 5.

Un PR abierto o aprobado todavía no es una tarea cerrada. Las correcciones se agregan con commits nuevos. Después del merge, actualizar avance y avisar a quienes quedaron desbloqueados; no reescribir el historial publicado.

5. Contratos, datos y comunicación

Cambiar un contrato sin romper el trabajo de otros

1. Abrir un issue que explique la interfaz actual, el cambio, la razón y a quién afecta.
2. Designar un autor y actualizar la sección correspondiente de `docs/07_especificaciones/00-contratos.md`. Los criterios de datos o comparación también se reflejan en su spec.
3. Obtener la aprobación de los otros dos y mergear el contrato antes del código dependiente.
4. Cada dueño adapta su módulo en su rama; coordinar el orden de los PR y los tests de integración.
5. Si cambian etiquetas, features, esquema o datos, identificar qué resultados pierden validez y cómo regenerarlos. Si hace falta reset, aplicarlo en una BD local con los resultados anteriores preservados cuando sea necesario.

Contratos sensibles: columnas de `datos_limpios`; firmas y claves de `evaluate` y `save_run`; interfaz de modelos; `NEEDS_SAMPLE_WEIGHT`; cálculo de pesos por fold; `feature_sets`; orden de clases y probabilidades.

Una base por persona

Cada integrante genera su propia `db/demanda.db`. La base local no se comparte mediante Git ni se usa como archivo editable entre los tres. Se comparten código, configuración, referencias de datos y comandos reproducibles. Las fixtures se identifican como ficticias y no se usan para presentar resultados reales.

Un cambio de etiqueta exige regenerar los datos y reentrenar los modelos afectados antes de volver a compararlos. Guardar solo el archivo anterior no vuelve comparables sus resultados. Para el docente, el equipo debe acordar un mecanismo explícito de acceso a la BD de entrega.

Tablero y límite de trabajo

Estados: **Backlog, En progreso, En revisión y Hecho**. Un bloqueo se marca en el issue con la dependencia, la persona de contacto y el próximo paso. Como referencia, mantener hasta dos tareas activas por persona y priorizar terminar o revisar antes de abrir más.

Mensajes que sirven para coordinar

Al tomar una tarea:

> Tomo ETL-4. Editaré `etl/load.py` y `tests/test_load.py`. Dependo de T-03. Próximo punto de entrega: PR para revisión de A. Si necesitás esos archivos, coordinemos acá antes de editarlos.

Al bloquearse:

> ETL-4 bloqueada: falta el esquema aprobado de T-03, a cargo de A. Puedo preparar tests contra el contrato. Necesito confirmar la firma y restricciones antes de integrar.

Al entregar:

> ETL-5 mergeada en PR #N. Validación y comandos en el PR. Desbloquea MC-1, MC-2 y ETL-6 de B, DB-1 de A y VIZ-1 de Angel Zambrano. Actualicen `main` y confirmen que pueden continuar.

Diariamente: indicar avance real, próxima tarea y bloqueo. Semanalmente: revisar dependencias, PR pendientes, compromisos de entrega y distribución del trabajo. El número de commits es una referencia; también cuentan revisión, integración, documentación y capacidad de explicar la solución. Si alguien estará ausente más de dos días, deja el estado y acuerda quién continúa.

6. Qué hacer cuando algo se traba

Situación	Acción y responsable
Falta una dependencia	Avisar en su issue, acordar un próximo punto de entrega y avanzar con trabajo independiente. Los stubs quedan en una rama de preparación; no sustituyen la integración final.
Dos personas necesitan el mismo archivo	Registrar autores, secciones y orden de integración. La segunda incorpora primero el PR de la primera. No resolverlo editando sin avisar.
ETL-5 se demora	Angel Zambrano reúne a los dueños de las piezas con fallos concretos. Priorizar pruebas e integración. Usar el ETL de referencia para preparar trabajo, indicando sus limitaciones; no presentar resultados definitivos con la fuga temporal pendiente.
XGBoost falla por clases o pesos	Angel Zambrano revisa el wrapper y coordina con B el runner y los pesos por fold. No cambiar clases, pesos o scoring de forma aislada.
Un fold no tiene todas las clases	B y el dueño del modelo verifican si el estimador puede entrenar. Documentar y acordar el tratamiento de CV o scoring. No ignorar errores de entrenamiento ni usar test para resolverlos.
Métricas inesperadamente altas	Revisar etiqueta, features, fechas, preprocesamiento y pesos. Evaluar la ablación y reproducir antes de interpretar una mejora.
`main` queda rota	Identificar el PR causante, avisar y preparar una corrección o un revert por PR. Si la CI impide la reparación, el mantenedor acuerda y registra cómo desbloquearla. Nunca force-push.
Se acerca la entrega	Los tres acuerdan prioridades y las registran en el plan. Primero una ejecución válida con modelos comparables, métricas, BD accesible, gráficos y defensa; después las ampliaciones.

Alcance y cierre

Los mínimos académicos y compromisos adicionales están en el plan SDD. Una demora no autoriza a eliminar modelos o funciones unilateralmente: acordar el cambio y actualizar el plan antes de seleccionar resultados.

El checklist general de entregables y sus estados se mantiene en avance, incluida la presentación, la entrega al campus, el acceso a la BD y la defensa. Esta guía no mantiene una segunda lista de casillas.